

WEB TECHNOLOGY

LABORATORY MANUAL

Complete Experiment Guide with Code, Output & Viva Preparation

Experiments 1 – 15 | HTML · CSS · JavaScript · PHP · React · XML

Table of Contents

Experiment 1	Basic HTML – Structure, Formatting, CV, Lists, Tables
Experiment 2	HTML Tables – College Timetable & Newspaper Layout
Experiment 3	HTML Forms – Registration Form
Experiment 4	HTML Forms – Form Fill-up Page
Experiment 5	XML & XSL – Bookstore with Stylesheet
Experiment 6	JavaScript – Prime Numbers & Calculator
Experiment 7	JavaScript – Form Validation
Experiment 8	JavaScript – Event Handling & Navigation Bar
Experiment 9	JavaScript – To-Do List Web Application
Experiment 10	PHP – OOP Abstract Class (BankAccount)
Experiment 11A	JavaScript – Event Handling & DOM Manipulation
Experiment 11B	JavaScript – Image Slider / Carousel
Experiment 12	JavaScript – Dynamic To-Do List
Experiment 13A	PHP – Dynamic Web Pages
Experiment 13B	PHP + AJAX – Delete MySQL Data Without Page Refresh
Experiment 14A	React.js – Building User Interfaces
Experiment 14B	React + Node.js – Backend Integration
Experiment 15A	XML – Storing and Transporting Data
Experiment 15B	PHP + XML – Reading XML Data in Webpage
Viva Q&A	Comprehensive Viva Question & Answer Bank

Experiment 1 – Basic HTML

1.1 – Basic HTML Page

Aim:

To create a basic HTML page with headings, paragraphs, links, and text formatting.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Assignment 1</title>
</head>
<body>
  <h1 align="center">HTML Assignment 1</h1>
  <hr color="red">
  <p>
    It is my <b>First HTML</b> Assignment.
    I am excited to learn HTML, and I want to make a career in
    <b>Web Designing</b>.
  </p>
  <h2>Introduction to HTML</h2>
  <p>
    HTML <i>(HyperText Markup Language)</i> is used to create documents
    on
    the <u><a href="https://www.google.com">World Wide Web</a></u>.
    It is simply a collection of documents displayed using a browser.
    HTML was developed by <strong>Tim Berners-Lee</strong> in 1992.
  </p>
</body>
</html>
```

1.2 – Text Formatting & Headings

Aim:

To demonstrate all HTML heading levels and text formatting tags including bold, italic, subscript, superscript, marquee, blockquote, and preformatted text.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Assignment 1.2</title>
</head>
<body>
  <marquee direction="right" style="background-color: pink">
    <h1 style="color: red"><em>Hey! This is me ABHIK!!!</em></h1>
  </marquee>
  <h1>This is an H1 Heading</h1>
  <h2>This is an H2 Heading</h2>
```

```

<h3>This is an H3 Heading</h3>
<h4>This is an H4 Heading</h4>
<h5>This is an H5 Heading</h5>
<h6>This is an H6 Heading</h6>
<p>This is a paragraph text.</p>
<hr color="red">
<p>This is the first line.<br>This is the second line.</p>
<blockquote>"This is a blockquote used to highlight important
text."</blockquote>
<pre>
This is preformatted text.
Line 1
Line 2
</pre>
<hr color="red">
<p><b>Bold</b> text</p>
<p><i>Italic</i> text</p>
<p><sub>Subscript</sub> text</p>
<p><sup>Superscript</sup> text</p>
<p><strong>Strong</strong> text</p>
<p><em>Emphasized</em> text</p>
<p><del>Deleted</del> text</p>
<p><ins>Inserted</ins> text</p>
<p><mark>Marked</mark> text</p>
</body>
</html>

```

1.3 – Biodata / CV Page

Aim:

To design a personal CV / biodata page using HTML with sections for personal info, education, skills, hobbies, and contact details.

Code:

```

<!DOCTYPE html>
<html>
<head>
  <title>My Biodata</title>
</head>
<body background="bg.jpg" style="background-size:cover">
  <h1 align="center"><font color="red">CURRICULUM VITAE</font></h1>
  <section align="center" style="color:pink">
    <h3>ABHIK DAS</h3>
    <p>
      Address: Ramkrishnapur, Barasat, West Bengal - 700124<br>
      <b>Mobile:</b> 6295434536<br>
      <b>Email:</b> abhikdas1066@gmail.com
    </p>
  </section>
  <hr>
  <h2>Career Objective</h2>
  <p>I am a motivated student eager to build a career in web
technology.</p>
  <h2>Education</h2>
  <ul>
    <li>B.Tech CSE (Data Science) - Brainware University</li>

```

```

        <li>Higher Secondary - 91.2%</li>
        <li>Secondary - 91.1%</li>
    </ul>
    <h2>Skills</h2>
    <ul>
        <li>Python</li>
        <li>C</li>
        <li>HTML</li>
    </ul>
    <h2>Hobbies</h2>
    <ul>
        <li>Watching Movies & Web Series</li>
        <li>Playing Cricket</li>
        <li>Listening to Music</li>
    </ul>
    <h2>Contact</h2>
    <ul>
        <li>Mobile: +91 6295434536</li>
        <li>Email: abhikdas1066@gmail.com</li>
        <li><a href="https://github.com/Abhik128911">GitHub Profile</a></li>
    </ul>
</body>
</html>

```

1.4 – Lists in HTML

Aim:

To demonstrate ordered and unordered nested lists in HTML.

Code:

```

<!DOCTYPE html>
<html>
<body>
    <p><b>Unordered List</b></p>
    <ul>
        <li>Shopping List
            <ul type="circle">
                <li>Tomatoes</li>
                <li>Bread</li>
                <li>Mushroom</li>
            </ul>
        </li>
    </ul>
    <p><b>Ordered List</b></p>
    <ol>
        <li>Morning Tasks
            <ol type="i">
                <li>Boil Water</li>
                <li>Make Tea</li>
                <li>Drink Tea</li>
            </ol>
        </li>
    </ol>
</body>
</html>

```

1.5 – Table in HTML

Aim:

To create an HTML table with border, caption, cellpadding, and student records.

Code:

```
<!DOCTYPE html>
<html>
<body>
  <table border="5" cellpadding="10" cellspacing="0">
    <caption>Student Record - Std XII</caption>
    <tr>
      <th>Roll No</th>
      <th>Name</th>
      <th>Address</th>
    </tr>
    <tr>
      <td>1</td>
      <td>Abhik</td>
      <td>Barasat</td>
    </tr>
    <tr>
      <td>2</td>
      <td>Ayusi</td>
      <td>Delhi</td>
    </tr>
    <tr>
      <td>3</td>
      <td>Rishi</td>
      <td>Kolkata</td>
    </tr>
  </table>
</body>
</html>
```

Experiment 2 – HTML Tables: Timetable & Newspaper Layout

2.1 – College Timetable

Aim:

To design a college timetable using HTML tables with rowspan and colspan attributes.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Assignment 2.1 - Time Table</title>
</head>
<body>
  <h1 align="center">COLLEGE TIME TABLE</h1>
  <table border="1" align="center" cellpadding="8" cellspacing="0">
    <tr>
      <th>Day</th>
      <th>8:30-9:30</th>
      <th>9:30-10:30</th>
      <th>10:30-11:30</th>
      <th>11:30-12:30</th>
      <th>12:30-2:00</th>
      <th>2:00-3:00</th>
      <th>3:00-4:00</th>
      <th>4:00-5:00</th>
    </tr>
    <tr>
      <th>Monday</th>
      <td>...</td>
      <td>SUB1</td>
      <td>SUB2</td>
      <td>SUB3</td>
      <td rowspan="6"><b>L<br>U<br>N<br>C<br>H</b></td>
      <td>SUB4</td>
      <td>SUB5</td>
      <td>COUNSELLING</td>
    </tr>
    <tr>
      <th>Tuesday</th>
      <td>SUB1</td><td>SUB2</td><td>SUB3</td><td>...</td>
      <td>SUB2</td><td>SUB2</td><td>LIBRARY</td>
    </tr>
    <tr>
      <th>Wednesday</th>
      <td>SUB1</td><td>SUB2</td><td>SWA</td><td>...</td>
      <td colspan="3"><b>LAB</b></td>
    </tr>
    <tr>
      <th>Thursday</th>
      <td>SUB1</td><td>SUB2</td><td>SUB3</td><td>...</td>
      <td>SUB2</td><td>SUB2</td><td>LIBRARY</td>
    </tr>
```

```

        <tr>
            <th>Friday</th>
            <td>SUB1</td><td>SUB2</td><td>SUB3</td><td>...</td>
            <td>SUB4</td><td>SUB5</td><td>LIBRARY</td>
        </tr>
        <tr>
            <th>Saturday</th>
            <td>SUB1</td>
            <td colspan="3"><b>SEMINAR</b></td>
            <td>SUB4</td><td>SUB5</td><td>LIBRARY</td>
        </tr>
    </table>
</body>
</html>

```

2.2 – Newspaper Layout Using Table

Aim:

To design a newspaper-style web page using HTML tables, multimedia (audio/video), and text formatting.

Code:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Assignment 2.2 - Newspaper</title>
</head>
<body>
    <h1 align="center">NEWSPAPER TITLE</h1>
    <table align="center" width="90%">
        <tr>
            <th colspan="3">
                <hr>
                TUESDAY, 5<sup>TH</sup> SEPTEMBER 2025
                <hr>
            </th>
        </tr>
        <tr>
            <td>
                <b>ARTICLE HEADING</b><br>
                <small>ABHIK DAS</small>
            </td>
            <td style="border-left:2px solid black; padding:10px; vertical-align:top">
                <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
                <audio controls>
                    <source src="audio.mp3" type="audio/mpeg">
                </audio>
            </td>
            <td style="border-left:2px solid black; padding:10px; vertical-align:top">
                <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
                <video controls width="250">
                    <source src="video.mp4" type="video/mp4">
                </video>
            </td>
        </tr>
    </table>

```



```

        </td>
    </tr>
    <tr>
        <td colspan="3" align="center">
            <b>"With great power comes great responsibility"</b>
        </td>
    </tr>
    <tr>
        <td>
            <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
            <h3>SUBHEADING</h3>
            <ul>
                <li>List 1</li>
                <li>List 2</li>
                <li>List 3</li>
            </ul>
        </td>
        <td style="border-left:2px solid black; padding:10px; vertical-align:top">
            <h3>SUBHEADING</h3>
            <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
        </td>
        <td style="border-left:2px solid black; padding:10px; vertical-align:top">
            <figure>
                
                <figcaption align="center">HOWRAH BRIDGE</figcaption>
            </figure>
        </td>
    </tr>
</table>
</body>
</html>

```

Experiment 3 – HTML Forms: Registration Form

Aim:

To design a user registration form using HTML form elements such as input, radio, checkbox, select, fieldset, and button.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Assignment 3.1 - Registration Form</title>
</head>
<body>
  <h2 align="center">Create a Free Account</h2>
  <center>
    <form action="/server" method="get">
      <!-- Account Details -->
      <fieldset>
        <legend>Account Details</legend>
        <table cellpadding="5">
          <tr>
            <td>Email*:<br><input type="email" name="email" size="35"
required></td>
            <td>Password*:<br><input type="password" name="password"
size="35" required></td>
          </tr>
          <tr>
            <td>Repeat Email*:<br><input type="email" name="reemail"
size="35" required></td>
            <td>Repeat Password*:<br><input type="password"
name="rpassword" size="35" required></td>
          </tr>
        </table>
      </fieldset>
      <!-- Personal Details -->
      <fieldset>
        <legend>Personal Details</legend>
        <table cellpadding="5">
          <tr>
            <td>Name*:<br><input type="text" name="name" size="35"
required></td>
            <td>Gender*:<br>
              <input type="radio" name="gender" value="male"> Male
              <input type="radio" name="gender" value="female"> Female
              <input type="radio" name="gender" value="other"> Other
            </td>
          </tr>
          <tr>
            <td>Phone No.*:<br><input type="number" name="phone"
required></td>
            <td>Birth Date*:<br>
              <select required>
                <option>1</option><!-- ...to 31 -->
              </select>
              <select required>
                <option>January</option><!-- ...to December -->
              </select>
            </td>
          </tr>
        </table>
      </fieldset>
    </form>
  </center>
</body>
</html>
```

```
</select>
<select required>
    <option>2001</option><!-- ...to 2030 -->
</select>
</td>
</tr>
<tr>
    <td>Address:<br><input type="text" size="35"></td>
    <td>Nationality*<br><input type="text" size="35"
required></td>
</tr>
<tr>
    <td>City*<br><input type="text" size="35"></td>
    <td>Marital Status*<br>
        <input type="radio" name="marital"> Married
        <input type="radio" name="marital"> Unmarried
    </td>
</tr>
<tr>
    <td>Country*<br><input type="text" size="35"></td>
    <td>Mobile*<br><input type="number" required></td>
</tr>
<tr>
    <td>Upload Your Photo*<br><input type="file" required></td>
</tr>
</table>
</fieldset>
<!-- Terms -->
<fieldset>
    <legend>Terms & Conditions*</legend>
    <input type="checkbox" required> I accept the terms &
conditions<br>
    <input type="checkbox" required> I agree to receive emails
</fieldset>
<br>
<center>
    <button type="reset">Reset</button>
    &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
    <button type="submit">Submit</button>
</center>
</form>
</center>
</body>
</html>
```

Experiment 4 – HTML Forms: Form Fill-up Page

Aim:

To create a detailed form fill-up page using HTML with various input types including text, number, radio, date, color, file, URL, and dropdown.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Fillup</title>
</head>
<body>
  <table border="2" width="500" cellpadding="7">
    <tr style="font-size:large"><td>Name</td>
      <td><input type="text" placeholder="Enter your name"></td></tr>
    <tr style="font-size:large"><td>Age</td>
      <td><input type="number" placeholder="Enter your age"></td></tr>
    <tr style="font-size:large"><td>Sex</td>
      <td>
        <input type="radio" id="male" name="gender" checked>
        <label for="male">Male</label>
        <input type="radio" id="female" name="gender">
        <label for="female">Female</label>
      </td></tr>
    <tr style="font-size:large"><td>Address</td>
      <td><textarea placeholder="Insert your
Address"></textarea></td></tr>
    <tr style="font-size:large"><td>Email</td>
      <td><input type="email" placeholder="Enter your email"></td></tr>
    <tr style="font-size:large"><td>Date of Birth</td>
      <td><input type="date"></td></tr>
    <tr style="font-size:large"><td>Height</td>
      <td><input type="number" value="150">cm</td></tr>
    <tr style="font-size:large"><td>Phone Number</td>
      <td><input type="tel" value="+91 "></td></tr>
    <tr style="font-size:large"><td>Favourite Color</td>
      <td><input type="color"></td></tr>
    <tr style="font-size:large"><td>FB, X, IG</td>
      <td><input type="url" placeholder="Enter URL"></td></tr>
    <tr style="font-size:large"><td>Passport size picture</td>
      <td><input type="file" accept="image/png, image/jpeg,
image/jpg"></td></tr>
    <tr style="font-size:large"><td>Select University</td>
      <td>
        <select name="university" style="width:100%; padding:5px; font-
size:16px;">
          <option value="" selected disabled>-- Select University --
</option>
          <option value="bwu">Brainware University</option>
          <option value="iit_kgp">IIT Kharagpur</option>
          <option value="ju">Jadavpur University</option>
          <option value="makaut">MAKAUT</option>
          <!-- ...more options -->
        </select>
      </td>
    </tr>
  </table>
```

```
        </td></tr>
    <tr>
        <td colspan="2" align="center" style="padding-top:20px;">
            <button type="submit" style="padding:10px 20px; border:none;
border-radius:4px; cursor:pointer; font-weight:bold;">
                Submit Application
            </button>
            <button type="reset" style="padding:10px 20px; border:none;
border-radius:4px; cursor:pointer; font-weight:bold; margin-left:10px;">
                Reset Form
            </button>
        </td>
    </tr>
</table>
</body>
</html>
```

Experiment 5 – XML & XSL: Bookstore

Aim:

To create an XML document for a bookstore with book details and write an XSL stylesheet to display those details in an HTML table.

XML Code (bookstore.xml):

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="Book_store.xsl"?>
<Bookstore>
  <Category type="Novel">
    <Book id="N1">
      <Title>God of Small Things</Title>
      <Author>Arundhati Roy</Author>
      <price currency="INR">450</price>
    </Book>
    <Book id="N2">
      <Title>Malgudi Days</Title>
      <Author>R.K Narayan</Author>
      <price currency="INR">450</price>
      <year>1997</year>
    </Book>
    <Book id="N3">
      <Title>Train to Pakistan</Title>
      <Author>Kushwant Singh</Author>
      <price currency="INR">1234</price>
    </Book>
  </Category>
  <Category type="Literature">
    <Book id="l1">
      <Title>Gitanjali</Title>
      <Author>Rabindranath Tagore</Author>
      <price currency="INR">250</price>
    </Book>
    <Book id="l2">
      <Title>Enemy of Aryavarta</Title>
      <Author>Anand</Author>
      <price currency="INR">420</price>
    </Book>
    <Book id="l3">
      <Title>The Great Gatsby</Title>
      <Author>F. Scott Fitzgerald</Author>
      <price currency="INR">5667</price>
    </Book>
  </Category>
</Bookstore>
```

XSL Stylesheet Code (Book_store.xsl):

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="html" encoding="UTF-8" indent="yes"/>
  <xsl:template match="/">
    <html>
      <head><title>Book Store</title></head>
```

```
<body>
  <h1>Welcome to the Book Store</h1>
  <table border="1">
    <tr bgcolor="#9acd32">
      <th>Title</th>
      <th>Author</th>
      <th>Price</th>
    </tr>
    <xsl:for-each select="Bookstore/Category/Book">
      <tr>
        <td><xsl:value-of select="Title"/></td>
        <td><xsl:value-of select="Author"/></td>
        <td><xsl:value-of select="price"/></td>
      </tr>
    </xsl:for-each>
  </table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>
```

Note: Open the XML file in a browser that supports XSLT (like Firefox). The browser will automatically apply the XSL stylesheet and render the table.

Experiment 6 – JavaScript: Prime Numbers & Calculator

6.1 – Prime Number Finder

Aim:

To write a JavaScript program that displays prime numbers within a user-specified range using prompt boxes.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Prime Number Finder</title>
</head>
<body>
  <h2>Prime Numbers in The Given Range</h2>
  <p>Check the result below after entering the range.</p>
  <button onclick="findprime()">Find The Prime Number</button>
  <div id="result"></div>
  <script>
    function findprime() {
      let start = parseInt(prompt("Enter the starting number:"));
      let end   = parseInt(prompt("Enter the ending number:"));
      let output = "<h3>Prime numbers between " + start + " and " + end
+ " are:</h3>";
      for (let i = start; i <= end; i++) {
        if (i < 2) continue;
        let isPrime = true;
        for (let j = 2; j <= Math.sqrt(i); j++) {
          if (i % j === 0) { isPrime = false; break; }
        }
        if (isPrime) output += i + "<br>";
      }
      document.getElementById("result").innerHTML = output;
    }
  </script>
</body>
</html>
```

6.2 – Simple Calculator

Aim:

To create a simple calculator using HTML and JavaScript that performs addition, subtraction, multiplication, and division, with proper handling of division by zero.

Code:

```
<!DOCTYPE html>
<html>
<head><title>Calculator</title></head>
<body>
  <h2>Calculator</h2>
  <label>Enter number 1:</label>
  <input type="number" id="num1"><br><br>
```



```

<label>Enter number 2:</label>
<input type="number" id="num2"><br><br>
<button onclick="add()">Add</button>
<button onclick="subtract()">Subtract</button>
<button onclick="multiply()">Multiply</button>
<button onclick="divide()">Divide</button>
<h3>Results:</h3>
<div id="result"></div>
<script>
    function getValues() {
        let a = parseFloat(document.getElementById("num1").value) || 0;
        let b = parseFloat(document.getElementById("num2").value) || 0;
        return { a, b };
    }
    function showResult(text) {
        document.getElementById("result").innerHTML += text + "<br>";
    }
    function add() { let {a,b} = getValues(); showResult(a + " + "
+ b + " = " + (a+b)); }
    function subtract() { let {a,b} = getValues(); showResult(a + " - "
+ b + " = " + (a-b)); }
    function multiply() { let {a,b} = getValues(); showResult(a + " × "
+ b + " = " + (a*b)); }
    function divide() {
        let {a,b} = getValues();
        if (b === 0) showResult("Cannot divide by 0");
        else showResult(a + " ÷ " + b + " = " + (a/b));
    }
</script>
</body>
</html>

```

Experiment 7 – JavaScript: Form Validation

Aim:

To create a registration form with client-side JavaScript validation for name (letters only), email (valid format), and a strong password (min 8 chars, uppercase, lowercase, digit, special character). Show appropriate error/success messages.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>
  <style>
    body { font-family: Arial; }
    .error { color: red; font-size: 14px; }
  </style>
</head>
<body>
  <h2>Registration Form</h2>
  <form onsubmit="return validateForm()">
    <label>Name:</label>
    <input type="text" id="name"><br>
    <span id="nameError" class="error"></span><br>

    <label>Email:</label>
    <input type="text" id="email"><br>
    <span id="emailError" class="error"></span><br>

    <label>Password:</label>
    <input type="password" id="password"><br>
    <span id="passError" class="error"></span><br><br>

    <button type="submit">Register</button>
  </form>
  <script>
    function validateForm() {
      let name      = document.getElementById("name").value.trim();
      let email     = document.getElementById("email").value.trim();
      let password  = document.getElementById("password").value.trim();

      let namePattern    = /^[A-Za-z ]+$/;
      let emailPattern   = /^[^ ]+@^[^ ]+\.[a-z]{2,3}$/;
      let passwordPattern = /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

      let valid = true;
      document.getElementById("nameError").innerHTML = "";
      document.getElementById("emailError").innerHTML = "";
      document.getElementById("passError").innerHTML = "";

      // Name Validation
      if (name === "") {
        document.getElementById("nameError").innerHTML = "Name is required";
        valid = false;
      }
    }
  </script>
</body>
</html>
```

```

    } else if (!namePattern.test(name)) {
        document.getElementById("nameError").innerHTML = "Only letters
allowed";
        valid = false;
    }

    // Email Validation
    if (email === "") {
        document.getElementById("emailError").innerHTML = "Email is
required";
        valid = false;
    } else if (!emailPattern.test(email)) {
        document.getElementById("emailError").innerHTML = "Invalid email
format";
        valid = false;
    }

    // Password Validation
    if (password === "") {
        document.getElementById("passError").innerHTML = "Password is
required";
        valid = false;
    } else if (!passwordPattern.test(password)) {
        document.getElementById("passError").innerHTML =
        "Password must contain 1 uppercase, 1 lowercase, 1 number, 1
special character and minimum 8 characters.";
        valid = false;
    }

    if (valid) alert("Registration Successful!");
    return valid;
}
</script>
</body>
</html>

```

Bug Fix: The original code had a typo — `document.getElementById(+ "passError")` — the `+` operator before the string caused a runtime error. Corrected to `document.getElementById("passError")`.

Experiment 8 – JavaScript: Event Handling & Navigation Bar

8.1 – Mouse Event Listener

Aim:

To create an HTML page with a button that uses `addEventListener()` to handle click, mouseover, mouseout, and dblclick events, changing the button's background color and displaying messages.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Mouse Event Listener</title>
  <style>
    div { align-items: center; text-align: center; margin-top: 50px; }
    #myBtn { width: 100px; height: 40px; }
  </style>
  <script>
    window.onload = function () {
      let btn = document.getElementById("myBtn");
      let demo = document.getElementById("demo");

      btn.addEventListener("click", function () {
        demo.innerHTML = "Button was clicked!";
        btn.style.backgroundColor = "green";
      });
      btn.addEventListener("mouseover", function () {
        demo.innerHTML = "Mouse is over the button!";
        btn.style.backgroundColor = "blue";
      });
      btn.addEventListener("mouseout", function () {
        demo.innerHTML = "Mouse left the button!";
        btn.style.backgroundColor = "red";
      });
      btn.addEventListener("dblclick", function () {
        demo.innerHTML = "Button double clicked!";
        btn.style.backgroundColor = "yellow";
      });
    };
  </script>
</head>
<body>
  <div>
    <h2>Mouse event Listener</h2>
    <button id="myBtn">Click Me</button>
    <p id="demo"></p>
  </div>
</body>
</html>
```

8.2 – Navigation Bar with Dropdown Toggle

Aim:

To create a navigation bar using HTML, CSS, and JavaScript where clicking the COURSES button toggles (show/hide) a dropdown list of courses.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Dropdown Example</title>
  <style>
    .navbar { background-color: black; padding: 15px; text-align:
center; position: relative; }
    .navbar button { background: none; border: none; color: white; font-
size: 16px; margin: 0 20px; cursor: pointer; }
    .navbar button:hover { color: yellow; }
    .dropdown { display: inline-block; position: relative; }
    #courseList {
      display: none; position: absolute; top: 35px; left: 0;
      background-color: black; color: white; width: 220px;
      max-height: 120px; overflow-y: auto; border: 1px solid white;
    }
    #courseList li { list-style: none; padding: 8px; cursor: pointer; }
    #courseList li:hover { background-color: gray; }
  </style>
  <script>
    function toggleCourses() {
      let list = document.getElementById("courseList");
      list.style.display = (list.style.display === "block") ? "none" :
"block";
    }
  </script>
</head>
<body>
  <div class="navbar">
    <button>HOME</button>
    <div class="dropdown">
      <button onclick="toggleCourses()">COURSES</button>
      <ul id="courseList">
        <li>Introduction to Programming</li>
        <li>Web Development</li>
        <li>Data Science</li>
        <li>Machine Learning</li>
      </ul>
    </div>
    <button>ABOUT US</button>
    <button>CONTACT US</button>
  </div>
</body>
</html>
```

Experiment 9 – JavaScript: To-Do List Web Application

Aim:

To create a To-Do List Web Application where the user can add, complete (toggle strikethrough), edit, and delete tasks dynamically using JavaScript DOM manipulation.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>To-Do List App</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f4f4f4;
text-align: center; margin-top: 50px; }
    .container { width: 400px; margin: auto; background: white; padding:
20px; border-radius: 10px; box-shadow: 0px 0px 10px gray; }
    input { width: 65%; padding: 8px; }
    button { padding: 6px 10px; margin-left: 5px; cursor: pointer;
border-radius: 5px; border: none; }
    ul { list-style-type: none; padding: 0; }
    li { background: #e3e3e3; margin: 10px 0; padding: 10px; border-
radius: 5px; display: flex; justify-content: space-between; align-items:
center; }
    .deleteBtn { background: red; color: white; }
    .editBtn { background: blue; color: white; }
    .completed { text-decoration: line-through; color: gray; }
    .btn-group button { margin-left: 5px; }
  </style>
</head>
<body>
  <div class="container">
    <h2>My To-Do List</h2>
    <input type="text" id="taskInput" placeholder="Enter a task">
    <button onclick="addTask()">Add</button>
    <ul id="taskList"></ul>
  </div>
  <script>
    function addTask() {
      var input = document.getElementById("taskInput");
      var taskText = input.value.trim();
      if (taskText === "") { alert("Please enter a task!"); return; }

      var li = document.createElement("li");
      var span = document.createElement("span");
      span.textContent = taskText;
      span.onclick = function () { span.classList.toggle("completed");
};

      li.appendChild(span);

      var btnGroup = document.createElement("div");
      btnGroup.className = "btn-group";

      var editBtn = document.createElement("button");
      editBtn.textContent = "Edit";
      editBtn.className = "editBtn";
      editBtn.onclick = function (e) {
```

```
        e.stopPropagation();
        var newTask = prompt("Update your task:", span.textContent);
        if (newTask !== null && newTask.trim() !== "") span.textContent
= newTask;
    };

    var deleteBtn = document.createElement("button");
    deleteBtn.textContent = "X";
    deleteBtn.className = "deleteBtn";
    deleteBtn.onclick = function (e) { e.stopPropagation();
li.remove(); };

    btnGroup.appendChild(editBtn);
    btnGroup.appendChild(deleteBtn);
    li.appendChild(btnGroup);
    document.getElementById("taskList").appendChild(li);
    input.value = "";
}
</script>
</body>
</html>
```

Experiment 10 – PHP: OOP Abstract Class (BankAccount)

Aim:

To create an abstract class BankAccount with a protected variable balance and an abstract method calculateInterest(). Implement child classes SavingAccount (5% interest) and CurrentAccount (2% interest) and display calculated interest for a balance of Rs. 10,000.

Code:

```
<?php
// Abstract class
abstract class BankAccount {
    protected $balance;

    function __construct($balance) {
        $this->balance = $balance;
    }

    // Abstract method - must be implemented by child classes
    abstract public function calculateInterest();
}

// Child class 1 - 5% interest rate
class SavingAccount extends BankAccount {
    public function calculateInterest() {
        $interest = $this->balance * 0.05;
        echo "Saving Account Interest: " . $interest . "<br>";
    }
}

// Child class 2 - 2% interest rate
class CurrentAccount extends BankAccount {
    public function calculateInterest() {
        $interest = $this->balance * 0.02;
        echo "Current Account Interest: " . $interest . "<br>";
    }
}

// Create objects with balance of 10000
$saving = new SavingAccount(10000);
$current = new CurrentAccount(10000);

// Call methods
$saving->calculateInterest(); // Output: Saving Account Interest: 500
$current->calculateInterest(); // Output: Current Account Interest: 200
?>
```

Expected Output:

Saving Account Interest: 500

Current Account Interest: 200

Experiment 11A – JavaScript Event Handling and DOM Manipulation

Aim:

To demonstrate JavaScript event handling using `addEventListener()` and DOM manipulation by changing heading text and paragraph content when a button is clicked.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Handling and DOM Manipulation</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 100px;
background: #f4f4f4; }
    #box {
      width: 300px; padding: 20px; margin: auto;
      background: white; border-radius: 10px;
      box-shadow: 0px 0px 10px gray;
    }
    button {
      padding: 10px 20px; margin-top: 20px; border: none;
      background: blue; color: white; border-radius: 5px; cursor:
pointer;
    }
    button:hover { background: darkblue; }
  </style>
</head>
<body>
  <div id="box">
    <h1 id="heading">Welcome!</h1>
    <p id="text">Click the button to change this text.</p>
    <button id="btn">Click Me</button>
  </div>
  <script>
    let button = document.getElementById("btn");
    button.addEventListener("click", function () {
      document.getElementById("heading").innerText = "Hello Student!";
      document.getElementById("text").innerText    = "Text changed using
DOM manipulation.";
    });
  </script>
</body>
</html>
```

Experiment 11B – JavaScript Image Slider (Carousel)

Aim:

To implement an image slider (carousel) using JavaScript that supports manual Previous/Next navigation and also auto-slides every 3 seconds.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Image Slider</title>
  <style>
    body { font-family: Arial; text-align: center; background: #f4f4f4;
margin-top: 50px; }
    .slider { width: 500px; margin: auto; position: relative; }
    img { width: 100%; height: 300px; border-radius: 10px; object-fit:
cover; }
    button {
      margin: 15px; padding: 10px 20px; border: none;
      background: blue; color: white; cursor: pointer; border-radius:
5px;
    }
    button:hover { background: darkblue; }
  </style>
</head>
<body>
  <h1>Image Slider (Carousel)</h1>
  <div class="slider">
    
  </div>
  <button onclick="prevImage()">Previous</button>
  <button onclick="nextImage()">Next</button>

  <script>
    let images = [
      "https://picsum.photos/id/1015/500/300",
      "https://picsum.photos/id/1016/500/300",
      "https://picsum.photos/id/1018/500/300",
      "https://picsum.photos/id/1020/500/300"
    ];
    let currentIndex = 0;

    function showImage() {
      document.getElementById("sliderImage").src = images[currentIndex];
    }
    function nextImage() {
      currentIndex = (currentIndex + 1) % images.length;
      showImage();
    }
    function prevImage() {
      currentIndex = (currentIndex - 1 + images.length) % images.length;
      showImage();
    }
  </script>
</body>
</html>
```

```
        // Auto slide every 3 seconds
        setInterval(nextImage, 3000);
    </script>
</body>
</html>
```

Experiment 12 – JavaScript: Dynamic To-Do List

Aim:

To create a dynamic To-Do List application using JavaScript where users can add tasks and delete them without page reload.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic To-Do List</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f4f4; display:
flex; justify-content: center; margin-top: 50px; }
    .container { background: white; padding: 25px; border-radius: 10px;
box-shadow: 0px 0px 10px gray; width: 400px; }
    h1 { text-align: center; color: #333; }
    .input-section { display: flex; gap: 10px; }
    input { flex: 1; padding: 10px; font-size: 16px; }
    button { padding: 10px 15px; background: green; color: white;
border: none; cursor: pointer; border-radius: 5px; }
    button:hover { background: darkgreen; }
    ul { list-style: none; padding: 0; margin-top: 20px; }
    li { background: #eee; padding: 12px; margin-bottom: 10px; display:
flex; justify-content: space-between; border-radius: 5px; }
    .delete-btn { background: red; padding: 5px 10px; border-radius:
5px; }
    .delete-btn:hover { background: darkred; }
  </style>
</head>
<body>
  <div class="container">
    <h1>To-Do List</h1>
    <div class="input-section">
      <input type="text" id="taskInput" placeholder="Enter a task">
      <button onclick="addTask()">Add</button>
    </div>
    <ul id="taskList"></ul>
  </div>
  <script>
    function addTask() {
      let taskInput = document.getElementById("taskInput");
      let taskValue = taskInput.value.trim();
      if (taskValue === "") { alert("Please enter a task!"); return; }

      let li = document.createElement("li");
      li.innerHTML = `${taskValue}
      <button class="delete-btn"
onclick="deleteTask(this)">Delete</button>`;
      document.getElementById("taskList").appendChild(li);
      taskInput.value = "";
    }
    function deleteTask(button) {
      button.parentElement.remove();
    }
  </script>
</body>
</html>
```

```
</script>  
</body>  
</html>
```

Experiment 13A – PHP: Dynamic Web Pages

Aim:

To create a dynamic web page using PHP that displays a personalised greeting, course name, today's date, and current time using PHP's built-in date() function.

Code (index.php):

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic PHP Web Page</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 80px;
background: #f4f4f4; }
    .box { width: 450px; margin: auto; padding: 25px; background: white;
border-radius: 10px; box-shadow: 0px 0px 10px gray; }
    h1 { color: blue; }
    p { font-size: 18px; }
  </style>
</head>
<body>
  <div class="box">
    <h1>Welcome to PHP Dynamic Page</h1>
    <?php
      $name    = "Student";
      $course  = "B.Tech";
      $date    = date("d-m-Y");
      $time    = date("h:i:s A");
      echo "<h2>Hello, $name!</h2>";
      echo "<p>Course: $course</p>";
      echo "<p>Today's Date: $date</p>";
      echo "<p>Current Time: $time</p>";
    ?>
  </div>
</body>
</html>
```

Experiment 13B – PHP + AJAX: Delete MySQL Data Without Page Refresh

Aim:

To delete records from a MySQL database using PHP and demonstrate how AJAX (XMLHttpRequest) can remove a table row from the UI without refreshing the entire page.

SQL Setup Code:

```
CREATE DATABASE studentdb;
USE studentdb;
CREATE TABLE students (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    name    VARCHAR(50),
    course  VARCHAR(50)
);
INSERT INTO students (name, course) VALUES
    ('Rahul', 'BCA'),
    ('Priya', 'B.Tech'),
    ('Amit', 'MCA');
```

Frontend Code (index.php):

```
<?php
$conn = mysqli_connect("localhost", "root", "", "studentdb");
$result = mysqli_query($conn, "SELECT * FROM students");
?>
<!DOCTYPE html>
<html>
<head>
    <title>Delete Data using AJAX</title>
    <script>
        function deleteData(id) {
            if (confirm("Are you sure?")) {
                let xhr = new XMLHttpRequest();
                xhr.open("GET", "delete.php?id=" + id, true);
                xhr.onload = function () {
                    if (this.status === 200) {
                        document.getElementById("row" + id).remove();
                    }
                };
                xhr.send();
            }
        }
    </script>
</head>
<body>
    <h1>Student Records</h1>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th><th>Name</th><th>Course</th><th>Action</th>
        </tr>
        <?php while ($row = mysqli_fetch_assoc($result)) { ?>
        <tr id="row<?php echo $row['id']; ?>">
            <td><?php echo $row['id'];      ?></td>
            <td><?php echo $row['name'];    ?></td>
```

```

        <td><?php echo $row['course']; ?></td>
        <td>
            <button onclick="deleteData(<?php echo $row['id']; ?>)">
                Delete
            </button>
        </td>
    </tr>
<?php } ?>
</table>
</body>
</html>

```

Backend Code (delete.php):

```

<?php
$conn = mysqli_connect("localhost", "root", "", "studentdb");
$id    = $_GET['id'];
mysqli_query($conn, "DELETE FROM students WHERE id = $id");
?>

```


Experiment 14A – React.js: Building User Interfaces

Aim:

To demonstrate using React.js (via CDN) to build an interactive user interface. The example uses React state (useState hook) to dynamically change text when a button is clicked.

Code (index.html):

```
<!DOCTYPE html>
<html>
<head>
  <title>React Text Change Example</title>
  <script
src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
  <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      const [text, setText] = React.useState("This is a simple React
example.");

      function changeText() {
        setText("Button Clicked Successfully!");
      }

      return (
        <div style={{ textAlign: "center", marginTop: "100px" }}>
          <h1>Welcome to React</h1>
          <p>{text}</p>
          <button onClick={changeText}>Click Me</button>
        </div>
      );
    }

    const root = ReactDOM.createRoot(document.getElementById("root"));
    root.render(<App />);
  </script>
</body>
</html>
```

Experiment 14B – React + Node.js: Backend Integration

Aim:

To demonstrate how a React.js frontend can fetch data from a Node.js/Express backend server using HTTP requests (Fetch API), simulating a real-world full-stack application.

How to Run

1. Install Node.js from <https://nodejs.org>
2. Create a folder e.g. ReactNodeProject
3. Inside the folder, save server.js and run: `npm install express cors`
4. Start server: `node server.js` (Terminal shows: Server running on `http://localhost:5000`)
5. Save index.html in the same folder and open it in a browser.

Backend Code (server.js):

```
const express = require("express");
const cors    = require("cors");
const app     = express();

app.use(cors());

const PORT = 5000;

// Sample data (simulates database data)
const students = [
  { id: 1, name: "Rahul", course: "BCA" },
  { id: 2, name: "Priya", course: "B.Tech" },
  { id: 3, name: "Amit", course: "MCA" }
];

// API route
app.get("/students", (req, res) => {
  res.json(students);
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Frontend Code (index.html):

```
<!DOCTYPE html>
<html>
<head>
  <title>React Fetch Data Example</title>
  <script
src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
  <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
```

```
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      const [students, setStudents] = React.useState([]);

      React.useEffect(() => {
        fetch("http://localhost:5000/students")
          .then(response => response.json())
          .then(data => setStudents(data));
      }, []);

      return (
        <div style={{ textAlign: "center", marginTop: "50px" }}>
          <h1>Student List from Backend</h1>
          {students.map(student => (
            <div key={student.id}>
              <p>{student.name} - {student.course}</p>
            </div>
          ))}
        </div>
      );
    }

    const root = ReactDOM.createRoot(document.getElementById("root"));
    root.render(<App />);
  </script>
</body>
</html>
```

Experiment 15A – XML: Storing and Transporting Data

Aim:

To implement an XML document for storing and transporting student data. XML (eXtensible Markup Language) uses user-defined tags to structure data in a human-readable and machine-readable format.

Code (students.xml):

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student>
    <id>1</id>
    <name>Rahul</name>
    <course>BCA</course>
    <city>Kolkata</city>
  </student>
  <student>
    <id>2</id>
    <name>Priya</name>
    <course>B.Tech</course>
    <city>Delhi</city>
  </student>
  <student>
    <id>3</id>
    <name>Amit</name>
    <course>MCA</course>
    <city>Mumbai</city>
  </student>
</students>
```

To view: Open the .xml file in any modern browser. The browser renders the XML tree in colour-coded format showing the hierarchy of elements.

Experiment 15B – PHP + XML: Reading XML Data in Webpage

Aim:

To read an XML file using PHP's `simplexml_load_file()` function and display the parsed data in an HTML table on a web page.

XML File (students.xml):

(Same as Experiment 15A — save in the same folder as `index.php`)

PHP Code (index.php):

```
<!DOCTYPE html>
<html>
<head>
    <title>XML Data using PHP</title>
    <style>
        body { font-family: Arial; text-align: center; margin-top: 50px;
background: #f4f4f4; }
        table { width: 70%; margin: auto; border-collapse: collapse;
background: white; }
        th, td { padding: 12px; border: 1px solid black; }
        th { background: lightblue; }
        h1 { color: blue; }
    </style>
</head>
<body>
    <h1>Student Data from XML using PHP</h1>
    <?php
        $xml = simplexml_load_file("students.xml");
        echo "<table>";
        echo "<tr>
            <th>ID</th>
            <th>Name</th>
            <th>Course</th>
            <th>City</th>
        </tr>";
        foreach ($xml->student as $student) {
            echo "<tr>";
            echo "<td>" . $student->id . "</td>";
            echo "<td>" . $student->name . "</td>";
            echo "<td>" . $student->course . "</td>";
            echo "<td>" . $student->city . "</td>";
            echo "</tr>";
        }
        echo "</table>";
    ?>
</body>
</html>
```

Viva Question & Answer Bank

The following Q&A covers all major topics from Experiments 1–15. Use this section to prepare for your lab viva examination.

Section 1 – HTML Fundamentals

Q1. What does HTML stand for?

Ans: HTML stands for HyperText Markup Language. It is the standard language used to create and structure content on the World Wide Web.

Q2. What is the difference between `<b>` and `<strong>`?

Ans: `<b>` makes text bold purely for visual/presentational purposes, while `<strong>` marks text as important with semantic meaning. Browsers display both as bold, but screen readers and SEO tools treat `<strong>` as meaningful emphasis.

Q3. What is the purpose of the DOCTYPE declaration?

Ans: `<!DOCTYPE html>` tells the browser which version of HTML the document uses. Without it, the browser may render the page in quirky/compatibility mode, causing inconsistent layout across browsers.

Q4. What is rowspan and colspan in HTML tables?

Ans: rowspan merges a cell vertically across multiple rows; colspan merges a cell horizontally across multiple columns. They are used to create complex table layouts such as timetables.

Q5. What are semantic HTML elements? Give examples.

Ans: Semantic elements clearly describe their meaning/purpose to both the browser and developer. Examples: `<header>`, `<footer>`, `<nav>`, `<article>`, `<section>`, `<main>`, `<aside>`. Non-semantic examples: `<div>`, `<span>`.

Q6. What is the difference between GET and POST methods in forms?

Ans: GET appends form data to the URL (visible, limited size, suitable for search/filter). POST sends data in the HTTP request body (not visible in URL, no size limit, suitable for sensitive data like passwords).

Q7. What input types were introduced in HTML5?

Ans: HTML5 added: email, url, number, range, date, time, month, week, color, search, tel. These provide built-in validation and mobile-friendly keyboards.

Q8. What is the use of the fieldset and legend tags?

Ans: fieldset groups related form controls visually with a box border. legend provides a caption/title for the fieldset group. Together they improve form organisation and accessibility.

Section 2 – CSS & Styling

Q1. What is the box model in CSS?

Ans: Every HTML element is a rectangular box made up of four layers from inside out: content (actual text/image), padding (space inside the border), border (line surrounding padding), and margin (space outside the border).

Q2. What is the difference between class and id selectors?

Ans: An id (#idName) must be unique on a page and has higher specificity. A class (.className) can be reused on multiple elements. Use id for unique elements (header, footer) and class for reusable styles.

Q3. What is the difference between position: absolute and position: relative?

Ans: relative positions an element relative to its own normal position in the document flow. absolute removes the element from the flow and positions it relative to its nearest positioned ancestor (one that is not static).

Q4. What is flexbox used for?

Ans: Flexbox (display: flex) is a CSS layout model for distributing space and aligning items in a container along a single axis (row or column). It simplifies tasks like centring, equal spacing, and responsive card layouts.

Section 3 – JavaScript

Q1. What is the DOM?

Ans: DOM stands for Document Object Model. It is a programming interface that represents the HTML document as a tree of objects. JavaScript can use the DOM to read, modify, add, or delete HTML elements and their attributes/styles dynamically.

Q2. What is the difference between innerHTML and innerText?

Ans: innerHTML reads/writes the HTML markup inside an element (including tags). innerText reads/writes only the plain text content, stripping all HTML tags. Use innerHTML when you want to insert HTML; innerText for plain text to avoid XSS risks.

Q3. What is event bubbling?

Ans: Event bubbling means that when an event is triggered on a nested element, it first runs the handler on that element, then bubbles up to each parent element in the DOM tree. Use event.stopPropagation() to stop the bubble.

Q4. What is the difference between == and === in JavaScript?

Ans: == (loose equality) compares values after type coercion (e.g., '5' == 5 is true). === (strict equality) compares both value AND type without coercion (e.g., '5' === 5 is false). Always prefer === to avoid unexpected bugs.

Q5. What are arrow functions in JavaScript?

Ans: Arrow functions (=>) are a shorter syntax for writing functions introduced in ES6. They do not have their own 'this' context. Example: const add = (a, b) => a + b; is equivalent to function add(a, b) { return a + b; }

Q6. What is the difference between var, let, and const?

Ans: var is function-scoped and can be re-declared; it is hoisted. let is block-scoped and cannot be re-declared in the same scope. const is block-scoped and must be initialised at declaration; its binding cannot be reassigned (though object properties can change).

Q7. What is a regex pattern? Give an example used in form validation.

Ans: A Regular Expression (regex) is a pattern used to match strings. Example: /^[A-Za-z]+\$/ matches strings containing only letters and spaces (used for name validation). /^(?=.*[A-Z])(?=.*[0-9]).{8,}\$/ checks for a strong password.

Q8. What is setInterval and how is it used in the image slider?

Ans: setInterval(callback, delay) repeatedly executes a function every 'delay' milliseconds. In the image slider, setInterval(nextImage, 3000) automatically advances the slide every 3 seconds without any user interaction.

Q9. What is addEventListener? How is it better than onclick?

Ans: addEventListener('event', handler) attaches multiple handlers to one element without overwriting previous ones. onclick = handler can only hold one function. addEventListener also supports the useCapture parameter for event capture/bubble control.

Q10. What is the purpose of event.stopPropagation() in the To-Do app?

Ans: In the To-Do List, the delete/edit buttons are inside a list item that also has an onclick handler (for toggle). Without stopPropagation(), clicking the button would trigger both the button's click and the parent li's click. stopPropagation() prevents the event from reaching the parent.

Section 4 – XML & XSL

Q1. What is XML and what is it used for?

Ans: XML (eXtensible Markup Language) is a markup language designed to store and transport data in a structured, human-readable format. Unlike HTML, XML tags are user-defined. It is commonly used in configuration files, web services (SOAP), and data exchange between systems.

Q2. What is the difference between XML and HTML?

Ans: HTML is used to display data in a browser and has predefined tags. XML is used to store/transport data with custom user-defined tags. XML is strict (case-sensitive, all tags must be closed), while HTML is more forgiving.

Q3. What is XSL and what is its role?

Ans: XSL (eXtensible Stylesheet Language) is used to transform and format XML documents. XSLT (XSL Transformations) converts XML into HTML, another XML, or plain text. The <?xml-stylesheet?> processing instruction links the XSL file to the XML document.

Q4. What is simplexml_load_file() in PHP?

Ans: simplexml_load_file() is a PHP function that reads an XML file and converts it into a SimpleXMLElement object. You can then access elements using object property syntax (e.g., \$xml->student->name) to iterate and display data in HTML.

Q5. What is a well-formed XML document?

Ans: A well-formed XML document must: have exactly one root element, have all tags properly closed, be case-sensitive (so <Name> and <name> are different), have all attributes quoted, and have no illegal characters in content without proper encoding.

Section 5 – PHP & MySQL

Q1. What is PHP and where is it executed?

Ans: PHP (Hypertext Preprocessor) is a server-side scripting language used to create dynamic web pages. PHP code runs on the web server (e.g., Apache via XAMPP), generates HTML output, and sends it to the browser. The client never sees PHP code.

Q2. What is the difference between echo and print in PHP?

Ans: Both output data. echo can output multiple comma-separated values and is slightly faster (no return value). print outputs one value and always returns 1, so it can be used in expressions. In practice, echo is more commonly used.

Q3. What is AJAX and why is it useful?

Ans: AJAX (Asynchronous JavaScript and XML) allows web pages to communicate with the server and update parts of the page without a full page reload. In Experiment 13B, clicking Delete removes the row immediately via XMLHttpRequest without refreshing the entire student list.

Q4. What is mysqli_connect() in PHP?

Ans: mysqli_connect(host, username, password, database) establishes a connection to a MySQL database. It returns a connection object used in subsequent queries. Always check the return value to handle connection errors gracefully.

Q5. What is OOP in PHP? What is an abstract class?

Ans: OOP (Object-Oriented Programming) in PHP uses classes to bundle data (properties) and behaviour (methods). An abstract class (declared with the abstract keyword) cannot be instantiated directly. It serves as a blueprint and can contain abstract methods that child classes must implement.

Q6. What is the difference between public, protected, and private in PHP?

Ans: public: accessible from anywhere (inside class, child class, outside). protected: accessible inside the class and its child classes only. private: accessible only inside the defining class. In Experiment 10, \$balance is protected so child classes (SavingAccount, CurrentAccount) can access it.

Q7. What is inheritance in PHP?

Ans: Inheritance allows a child class to extend a parent class using the 'extends' keyword, inheriting all non-private properties and methods. The child class can override parent methods or add new ones. Example: SavingAccount extends BankAccount.

Q8. What is the use of the date() function in PHP?

Ans: date(format) returns the current date and time as a formatted string. Common format characters: d (day), m (month), Y (4-digit year), h (12-hour), H (24-hour), i (minutes), s (seconds), A (AM/PM). Example: date('d-m-Y') returns '20-05-2026'.

Section 6 – React.js & Node.js

Q1. What is React.js?

Ans: React.js is an open-source JavaScript library (developed by Meta/Facebook) for building user interfaces. It uses a component-based architecture where the UI is split into reusable components. React uses a Virtual DOM to efficiently update only the changed parts of the actual DOM.

Q2. What is JSX?

Ans: JSX (JavaScript XML) is a syntax extension for JavaScript that looks like HTML inside JavaScript code. Babel compiles JSX to regular JavaScript (React.createElement calls). JSX makes it easier to write and understand the structure of React components.

Q3. What is a React state?

Ans: State is a built-in React object that stores data that can change over time. When state changes, React automatically re-renders the component. The useState hook returns the current state value and a setter function: `const [text, setText] = React.useState('initial value');`

Q4. What is the useEffect hook in React?

Ans: `useEffect(callback, dependencies)` runs side effects (like data fetching, subscriptions) after rendering. With an empty dependency array `[]`, it runs only once after the initial render — equivalent to `componentDidMount` in class components. In Experiment 14B, it fetches student data from the backend on page load.

Q5. What is CORS and why was it needed in Experiment 14B?

Ans: CORS (Cross-Origin Resource Sharing) is a browser security policy that blocks requests from one origin (e.g., `file://` or `localhost:3000`) to a different origin (`localhost:5000`). By adding `app.use(cors())` in the Express server, the server sends headers allowing the frontend to access the API.

Q6. What is Node.js and Express.js?

Ans: Node.js is a JavaScript runtime that allows JavaScript to run on the server side (outside the browser), built on Chrome's V8 engine. Express.js is a minimal web framework for Node.js that simplifies creating REST APIs and web servers. `app.get('/route', handler)` defines a route.

Q7. What is the Fetch API?

Ans: The Fetch API is a modern browser API for making HTTP requests (replacing `XMLHttpRequest`). `fetch(url)` returns a Promise. You chain `.then(res => res.json())` to parse the JSON response. In Experiment 14B, `fetch('http://localhost:5000/students')` retrieves student data from the Node.js server.

— End of Lab Manual —